

Documentación Técnica - EdgeLeak AI (Fase MVP)

1. Arquitectura del Sistema

El proyecto está desarrollado bajo el patrón de diseño de software **MVC (Modelo-Vista-Controlador)**, adaptado a las convenciones de reactividad de Flutter mediante el uso de `ChangeNotifier` y `ListenableBuilder`. Esta arquitectura garantiza un bajo acoplamiento y una alta cohesión, separando estrictamente la interfaz gráfica de la lógica de negocio y el acceso a datos.

La estructura del código (`lib/`) se divide en:

- **Config (`/config`):** Contiene la configuración global, enrutamiento estático (`AppRoutes`) y la paleta de colores semánticos (`Material Design 3`) para los estados del sistema.
- **Modelos (`/data/models`):** Estructuras de datos puras en Dart (POJOs) que mapean las entidades del dominio (`UsuarioModel`, `AlertaFugaModel`, `LecturaSensorModel`) e incluyen métodos de serialización y deserialización (`JSON/Map`) para la comunicación con APIs y bases de datos.
- **Servicios (`/data/services`):** Capa de infraestructura responsable de las transacciones asíncronas externas (Consultas HTTP REST a Groq API y transacciones SQL locales vía `Sqflite`).
- **Controladores (`/controllers`):** El núcleo lógico de la aplicación. Gestionan el estado de la sesión (`AuthController`) y la lógica matemática del simulador IoT (`DashboardController`).
- **Vistas (`/view`):** Pantallas sin estado lógico complejo, encargadas puramente de la representación visual y de reaccionar a los cambios notificados por los controladores.

2. Stack Tecnológico

- **Framework Frontend:** Flutter (SDK) / Lenguaje: Dart.
- **Inteligencia Artificial:** Groq API (Modelo Llama-3.3-70B-Versatile) consumido vía HTTP REST.
- **Persistencia Local:** SQLite (mediante el paquete `sqflite`).
- **Seguridad y Entorno:** Manejo de credenciales encriptadas mediante `flutter_dotenv`.
- **Testing:** `flutter_test` (Desarrollo guiado por pruebas - TDD para validaciones lógicas).

3. Esquema de Base de Datos Relacional (Local SQLite)

El sistema implementa una base de datos embebida (`edgeleak_v2.db`) para garantizar el funcionamiento offline parcial y la persistencia de datos.

Tabla 1: usuarios

- `id` (INTEGER, PRIMARY KEY, AUTOINCREMENT)
- `nombre` (TEXT)
- `correo` (TEXT, UNIQUE) - Llave de negocio para evitar duplicidad.
- `password` (TEXT) - Nota: En fases futuras se implementará hashing (ej. `bcrypt`).

Tabla 2: historial

- id (INTEGER, PRIMARY KEY, AUTOINCREMENT)
- veredicto (TEXT) - Clasificación de la IA (Ej. "Fuga Detectada").
- severidad (TEXT) - Nivel de criticidad ("Normal", "Advertencia", "Crítica").
- mensaje (TEXT) - Acción recomendada generada por la IA.
- fecha (TEXT) - Timestamp en formato ISO-8601.

4. Módulo de Simulación de Hardware (Gemelo Digital)

Ante la ausencia de hardware físico en la fase MVP, se desarrolló un simulador dinámico en el DashboardController. Este módulo utiliza funciones de la librería matemática de Dart (dart:math) para inyectar fluctuaciones de caudal en tiempo real simulando el sensor YF-S201:

- **Estado Normal:** Fluctuación entre 0.1 y 0.5 L/min.
- **Estado Anomalía (Goteo/Obstrucción):** Fluctuación entre 1.2 y 2.5 L/min.
- **Estado Fuga Crítica (Ruptura):** Fluctuación entre 6.0 y 9.0 L/min.

Visualmente, estas fluctuaciones se representan mediante un CustomPainter (Animación 3D del agua), el cual renderiza ondas senoidales matemáticas (

$$y=A\cdot\sin(\omega x+\phi)$$

) que alteran su amplitud y velocidad dependiendo de la criticidad del evento simulado.

5. Integración de Inteligencia Artificial (Prompt Engineering)

El sistema no utiliza condicionales estáticos (if/else) para clasificar fugas, sino que delega el análisis al modelo de IA **Llama 3.3 70B** de Groq.

La integración se realiza mediante una petición POST configurada estrictamente para retornar un json_object. El Payload estructurado incluye:

1. **System Role:** Define el comportamiento del modelo y el esquema JSON exacto esperado (Veredicto, Severidad, Mensaje).
2. **User Role:** Inyecta la lectura de caudal en tiempo real (

L/minL/min

) y los parámetros de tolerancia del sistema métrico.
3. **Temperature (0.0):** Se anuló la "creatividad" del LLM (temperatura en 0) para garantizar respuestas deterministas y evitar alucinaciones, asegurando que la estructura JSON no se rompa al deserializar en la aplicación.

6. Módulo de Seguridad y Autenticación

La aplicación incluye un sistema de registro con validación en tiempo real de expresiones regulares (Regex) para políticas de contraseñas fuertes:

- Longitud mínima de 8 caracteres.
- Inclusión obligatoria de letras mayúsculas [A-Z].
- Inclusión obligatoria de valores numéricos [0-9].
- Inclusión obligatoria de caracteres especiales [!@#\$%^&*~].

Además, incluye un algoritmo criptográfico básico (`generarContraseñaSegura()`) para sugerir a los usuarios claves robustas y aleatorias utilizando métodos de shuffle sobre matrices de caracteres.